

Пути в графах

Расстоянием $d(x, y)$ между двумя связанными вершинами x, y называется **длина наименьшего пути между ними** (понятно, что такой путь обязательно является простым).

В том случае, когда вершины не являются связанными, полагают, что $d(x, y) = \infty$

Диаметром графа $diam(G)$ называется максимальное расстояние между его вершинами.

В случае несвязного графа полагают, что $diam(G) = \infty$.

Эксцентриситетом вершины называется максимальное расстояние от нее до любой вершины графа.

Радиусом графа $r(G)$ называется минимальный из эксцентриситетов вершин графа G , а вершины, на которых этот минимум достигается, называются **центральными вершинами**.

Множество всех центральных вершин называется **центром графа**.

Характеризация двудольных графов в терминах циклов

Теорема (Кениг)

Граф G является двудольным тогда и только тогда, когда в нем отсутствуют циклы нечетной длины (нечетные циклы).

Доказательство.

1. Граф G двудольный $\rightarrow$ нет нечетных циклов

Действительно, для того чтобы, выйдя из произвольной вершины x блока X , затем в нее же и вернуться, нам необходимо сделать четное число шагов.

Следовательно, любой цикл в двудольном графе G обязан иметь четную длину

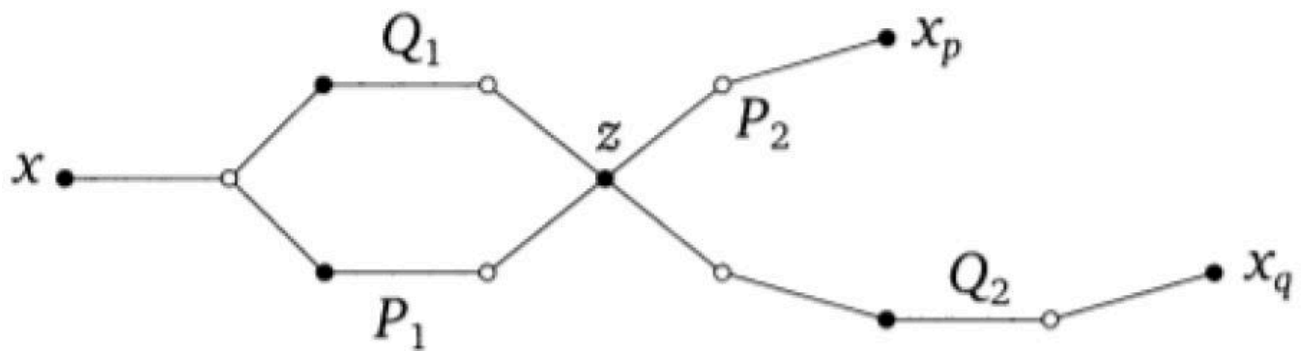
2. Нет нечетных циклов $\rightarrow$ граф G двудольный (достаточно ограничиться связными графами, так как любой несвязный граф является двудольным тогда и только тогда, когда двудольной является любая из его компонент связности)

$x \in V$ – произвольная вершина

Блок Y : все вершины y , длина кратчайшего пути до которых от x нечетная

Блок X : остальные

Докажем, что любые две вершины x_p, x_q , из подмножества X смежными не являются.



Рассмотрим произвольные кратчайшие пути P и Q , соединяющие эти вершины с вершиной x .

z – последняя общая вершина этих путей.

z разделяет пути P и Q на два подпути:

1. участки P_1 и Q_1 от x до z
2. подпути P_2 и Q_2 , соединяющие z с вершинами x_p и x_q соответственно.

Длины путей P_1 и Q_1 обязаны совпадать – в противном случае можно было бы сократить путь от x до одной из двух вершин x_p, x_q что невозможно.

Так как x_p и x_q принадлежат блоку X , длины путей P и Q имеют одинаковую четность, следовательно, одинаковую четность имеют и участки P_2 и Q_2 путей P и Q .

Кроме того, P_2 и Q_2 не имеют никаких других общих вершин, помимо z . Добавление к ним ребра $e = \{x_p, x_q\}$ приводит к образованию в G цикла нечетной длины, чего быть не может.

Аналогично доказывается, что и любые две вершины y_p, y_q блока Y смежными не являются. Следовательно, граф G является двудольным.

Орграф

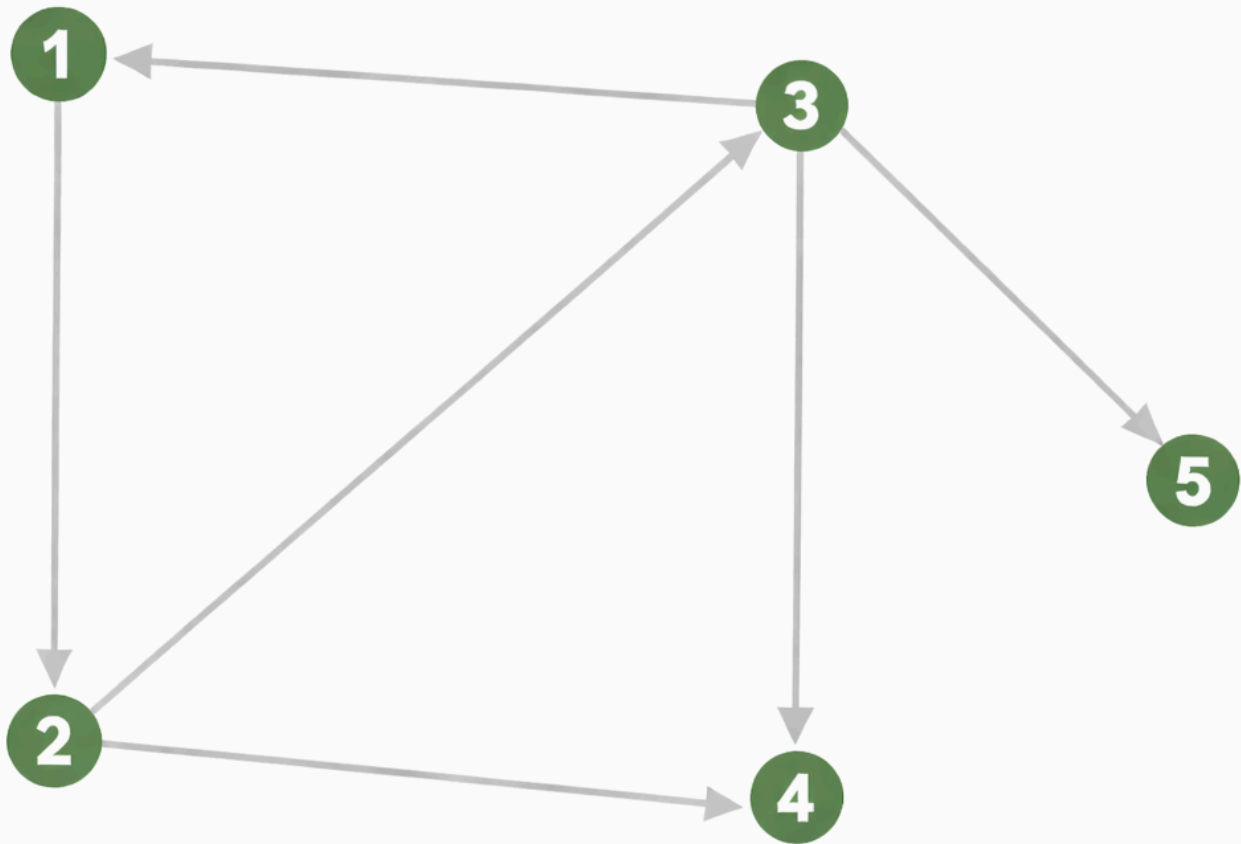
Маршрут, путь, простой путь, цикл

Вершины x и y орграфа называются связанными, если существует хотя бы один путь из x в y и хотя бы один путь из y в x .

Отношение связности – отношение эквивалентности. **Классы эквивалентности** – компоненты сильной связности.

Орграф называется **сильно связным**, если он состоит из единственной компоненты сильной связности.

Орграф называется **слабо связным**, если при замене всех дуг на ребра (неориентированные) получившийся неориентированный граф является связным.



Компоненты этого графа: $\{1, 2, 3\}\{4\}\{5\}$

Лемма. Пусть F и H две различные компоненты сильной связности орграфа D и пусть существует дуга e из F в H . Тогда дуги из H в F отсутствуют.

По любому орграфу D можно построить так называемый граф $C(D)$ компонент сильной связности графа D , вершинами которого будут компоненты сильной связности графа D , а дугами – дуги графа D , направленные из одной компоненты в другую.

Теорема. В орграфе $C(D)$ циклы отсутствуют, то есть он является **ациклическим орграфом** (DAG – directed acyclic graph).

Доказательство.

Предположим что существует цикл их трех вершин F , G и H , то все вершины

вовлеченные в этот цикл, находясь в исходном графе, образуют единую компоненту связности, что противоречит нашему условию.

Способы представления графа в компьютере

Матрица смежности

Пусть у нас есть граф $G = (V, E)$, $|V| = n$, $|E| = m$. $A : n \times n$

$$a_{ij} = \begin{cases} 1, & \exists(i, j) \\ 0, & \nexists(i, j) \end{cases}$$

Преимущество: за константу отвечаем на вопрос "Смежны ли данные вершины?"

Недостаток: расходуется много памяти, нужно иметь n^2 ячеек памяти. Если граф разреженный, то большинство этих ячеек будет занято нулями.

Лучше использовать при плотном или небольшом по числу вершин графе.

Список смежности

Когда для каждой вершины существует список смежных с ней вершин.

Преимущество: эффективно используется память, экономим время на обращение

Недостаток: чтобы ответить на вопрос: "Смежны ли данные вершины?", нужно перебрать соответствующий список смежности.

Матрица инцидентности

В этой матрице столбцы соответствуют ребрам, а строки вершинам. В случае неориентированного графа – в столбце соответствующей дуге (i, j) 1 ставят на пересечении строке i и j , а в случае орграфа в столбце дуге (i, j) на пересечении строкой i стоит -1, а j стоит 1. Все другое – это нули.

Недостаток: матрица по большей части содержит нули

Списки инцидентности (ребер)

Списки ребер.

Взвешенный граф

Любому ребру (дуге), соответствует вещественное число, называемое весом, данного ребра или дуги. Используется матрица весов $A : n \times n$.

$$a_{ij} = \begin{cases} \text{вес}(i, j), & \exists(i, j) \\ \infty, & \nexists(i, j) \end{cases}$$

Алгоритмы обходов

Обойти граф – побывать в каждой его вершине. Существует два базовых подхода: обход в ширину и в глубину. В базовом виде алгоритмы обхода казалось бы не несут особой практической пользы, но на самом деле они очень важны, потому что являются основой для множества различных алгоритмов. Существует рекурсивный и нерекурсивный вариант для обхода в глубину, для алгоритма обхода в ширину обычно используется нерекурсивный вариант. Существует еще два подхода для реализации этих алгоритмов с использованием двух и трех цветов. Если в каких-то случаях все равно, что использовать то бывают случаи когда двух цветов недостаточно.

Обход в глубину

Убираем начальную вершину и проходим от нее вперед настолько насколько можем, то есть находим смежные с ней (непросмотренную) потом смежную с этой (непросмотренная), пока у очередной вершины не окажется смежных (непросмотренных) в этом случае возвращаемся на шаг назад.

Непросмотренные вершины – так как граф является нелинейной структурой, то нам приходится периодически возвращаться и искать новые пути, чтобы не запутаться и не ходить к одному и тому же маршруту, мы должны помечать те места где мы уже были.

Рекурсивный обход в глубину

Массив `used[1:n]` (пройденные вершины)

Граф задается списками смежности `Adj[v]`

До начала работы алгоритма

```
for (v in V)
    used[v] = 0;
```

```
void dfs(v)
    used[v] = 1;
    for u in Adj[v]`
        if (!used[u])
            dfs(u);
```

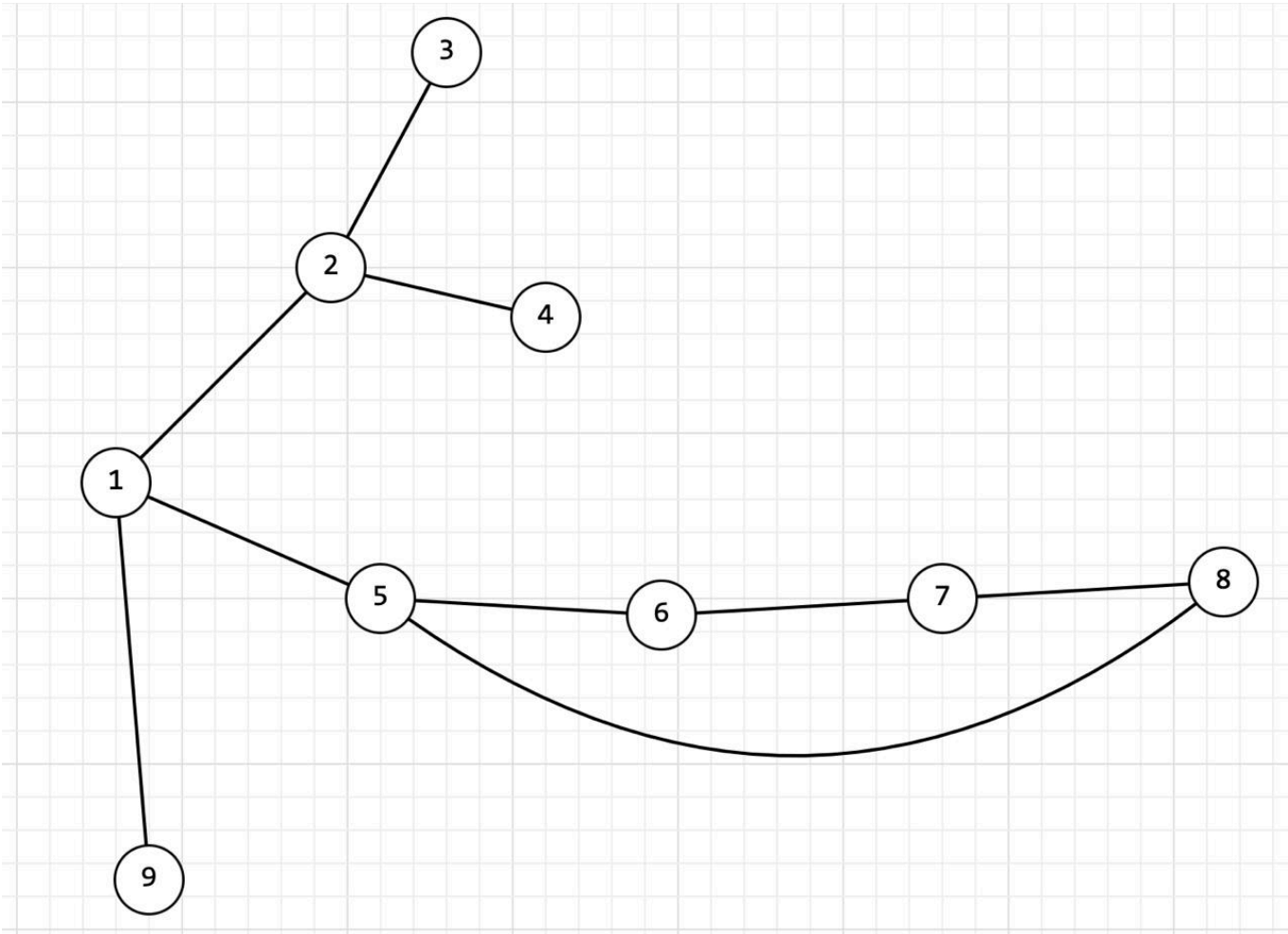
Для несвязного графа запускается в цикле

```
for (v in V)
    if (!used[v])
        dfs(v);
```

Нерекурсивный обход в глубину (на экзамене)

Для того чтобы перейти к рекурсии к нерекурсии нужно использовать какую-то вспомогательную структуру в которой вы будете хранить вершины к которым вы будете возвращаться в ходе проведения обхода, так как в случае обхода в глубину возвращение всегда идет на шаг назад, то логично использовать стек.

```
void dfs(v)
    used[v] = 1; push(s, v); # s – стек
    while (!empty(s))
        top(s, u);
        if (∃w in Adj[u]: used[w] = 0) # цикл
            used[w] = 1;
            push(s, w);
        else:
            pop(s);
```



used	стек	экран/файл
000000000	пуст	пуст
100000000	1	1

used	стек	экран/файл
110000000	21	1, 2
111000000	321	1, 2, 3
- -	21	- -
111100000	421	1, 2, 3, 4
- -	21	- -
-	1	- -
111110000	51	1, 2, 3, 4, 5
111111000	651	1, 2, 3, 4, 5, 6
111111100	7651	1, 2, 3, 4, 5, 6, 7
111111110	87651	1, 2, 3, 4, 5, 6, 7, 8
- -	7651	- -
- -	651	- -
- -	51	- -
- -	1	- -
111111111	91	1, 2, 3, 4, 5, 6, 7, 8, 9
- -	1	- -
- -	пуст	- -

Обход в глубину (три цвета)

Обход в глубину (три цвета)

Цвета вершин – белый, серый, черный (массив color)

Метки времени (d - вершина и открывается (становится серой), f - вершина и закрывается (становится черной))

Массив предшественников $\pi[u]$

```

DFS(G)
  for каждой вершины  $u$  in  $V[G]$ 
    do color[u] ← WHITE
       $\pi[u]$  ← NULL
  time ← 0
  for каждой вершины  $u$  in  $V[G]$ 
    do if color[u] = WHITE
       then DFS_Visit(u)
DFS_Visit(u)
  color[u] ← GRAY // открыта белая вершина  $u$ 

```

```

time ← time + 1
d[u] ← time
for каждой вершины v in Adj[u] // исследование ребра (u, v)
    do if color[v] = WHITE
        then π[v] ← u
            DFS_Visit(v)
color[u] ← BLACK // завершение
f[u] ← time ← time + 1

```

```

DFS(G)
    for каждой вершины u in G.V
        u.color = WHITE
        u.π = NULL
    time = 0
    for каждой вершины u in G.V
        if color[u] == WHITE
            DFS_Visit(G, u)
DFS_Visit(G, u)
    time = time + 1 // открыта белая вершина u
    u.d = time
    u.color = GRAY
    for каждой вершины v in G.Adj[u] // исследование ребра (u, v)
        if v.color == WHITE
            v.π = u
            DFS_Visit(G, v)
    u.color = BLACK // завершение работы с u
    time = time + 1
    u.f = time

```

Подграф предшествования

Рассмотрим подграф предшествования обхода в глубину

$$G_\pi = (V, E_\pi), E_\pi = \{(\pi[u], u) : u \in V, \pi[u] \neq NULL\}$$

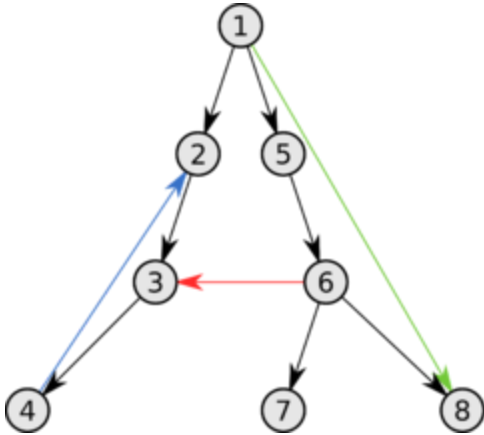
Подграф предшествования поиска в глубину образует лес обхода в глубину, который состоит из нескольких деревьев обхода в глубину. С помощью полученного леса можно классифицировать ребра графа G .

Ребрами дерева назовем те ребра из G , которые вошли в G ,

Ребра (u, v) , соединяющие вершину u с её предком v в дереве обхода в глубину назовем **обратными ребрами** (для неориентированного графа предок должен быть не родителем, так как иначе ребро будет являться ребром дерева).

Ребра (u, v) , не являющиеся ребрами дерева и соединяющие вершину u с её потомком v в дереве обхода в глубину назовем **прямыми ребрами** (в неориентированном графе нет разницы между прямыми и обратными ребрами, поэтому все такие ребра считаются

обратными). Все остальные ребра назовем **перекрестными ребрами** – такие ребра могут соединять вершины одного и того же дерева обхода в глубину, когда ни одна из вершин не является предком другой, или соединять вершины в разных деревьях.



Черным цветом отмечены – ребра дерева, синим – обратные ребра, зеленым – прямые ребра, красным – перекрестные ребра.

Алгоритм dfs можно модифицировать так, что он будет классифицировать встречающиеся при работе ребра. Идея состоит в том, что каждое ребро (u, v) можно классифицировать при помощи цвета вершины и при первом его исследовании, а именно: Белый цвет вершины v по определению dfs говорит о том, что это ребро дерева. Серый цвет в силу того, что серые вершины всегда образуют нисходящий путь в каком-либо из деревьев dfs и встреченная вершина u лежит на нем выше вершины u , определяет обратное ребро (для неориентированного графа необходимо проверить условие $v \neq \pi[u]$).

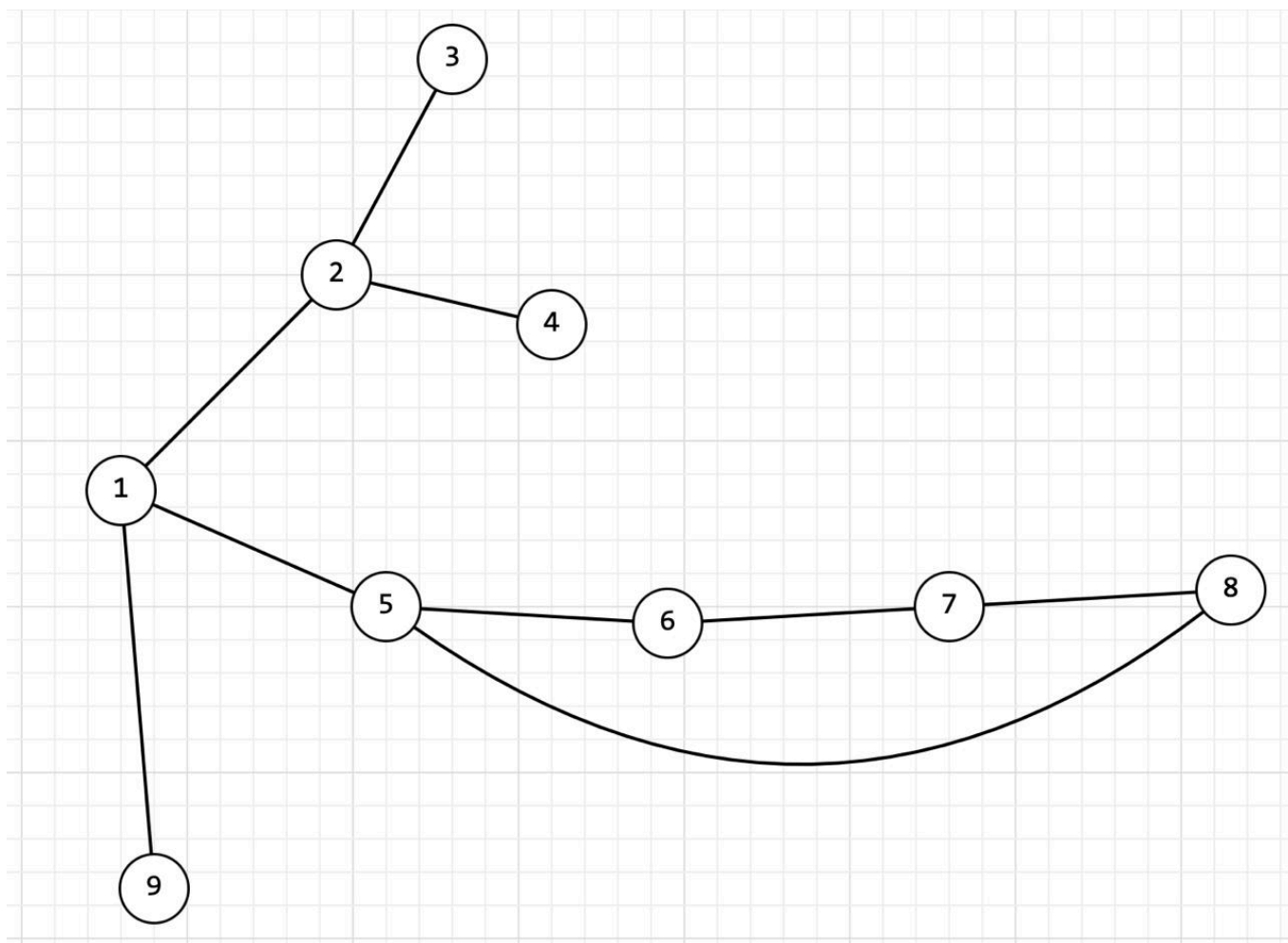
Черный цвет, соответственно, указывает на прямое или перекрестное ребро.

Замечание. Можно доказать, что прямые и перекрестные ребра никогда не встречаются при поиске в глубину в неориентированном графе.

Обход в ширину

Вспомогательная структура – очередь, так как мы обрабатываем сразу все вершины на данном уровне.

```
void bfs(v)
    used[v] = 1; push(q, v)
    while (!empty(q))
        pop(q, u)
        for (w in Adj[u])
            if (!used[w])
                used[w] = 1
                push(q, w)
```



used	очередь	экран/файл
000000000	пуста	пуст
100000000	1	1
110010001	259	1, 2, 5, 9
111110001	5924	1, 2, 5, 9, 3, 4
- -	3468	- -
111111111	87	1, 2, 3, 4, 5, 6, 7, 8, 9
- -	7	- -
- -	пуста	- -

Обход в ширину (три цвета)

π – предшественник, color – цвет, enqueue – вытащить из очереди, dequeue – добавить в очередь.

```

BFS(G, s)
  for каждой вершины u in V[G] - s
    do color[u] ← WHITE
       d[u] ← ∞
       π[u] ← NULL
  color[s] ← GRAY
  d[s] ← 0
  π[s] ← NULL
  Q ← 0
  Enqueue(Q, s)
  while Q ≠ 0
    do u ← Dequeue(Q)
       for каждой v in Adj[u]
         do if color[v] = WHITE
              then color[v] ← GRAY
                 d[v] ← d[u] + 1
                 π[v] ← u
                 Enqueue(Q, v)
  color[u] ← BLACK

```

Д/з

1. Докажите, что в любом графе G расстояние $d(x, y)$ между вершинами удовлетворяет неравенству треугольника: $d(x, z) + d(z, y) \geq d(x, y)$.
2. Докажите, что радиус и диаметр графа связаны следующим образом:

$$r(G) \leq \text{diam}(G) \leq 2r(G).$$
3. Узнать для чего используется матрица инцидентности (дописать преимущество), в общем и конкретные примеры.
4. Проанализировать два варианта нерекурсивного обхода в глубину. Почему первый вариант лучше?

```

void dfs(v)
  used[v] = 1; push(s, v); # s — стек
  while (!empty(s))
    top(s, u);
    if (w in Adj[u]: used[w] = 0) # цикл
      used[w] = 1;
      push(s, w);
    else:
      pop(s);

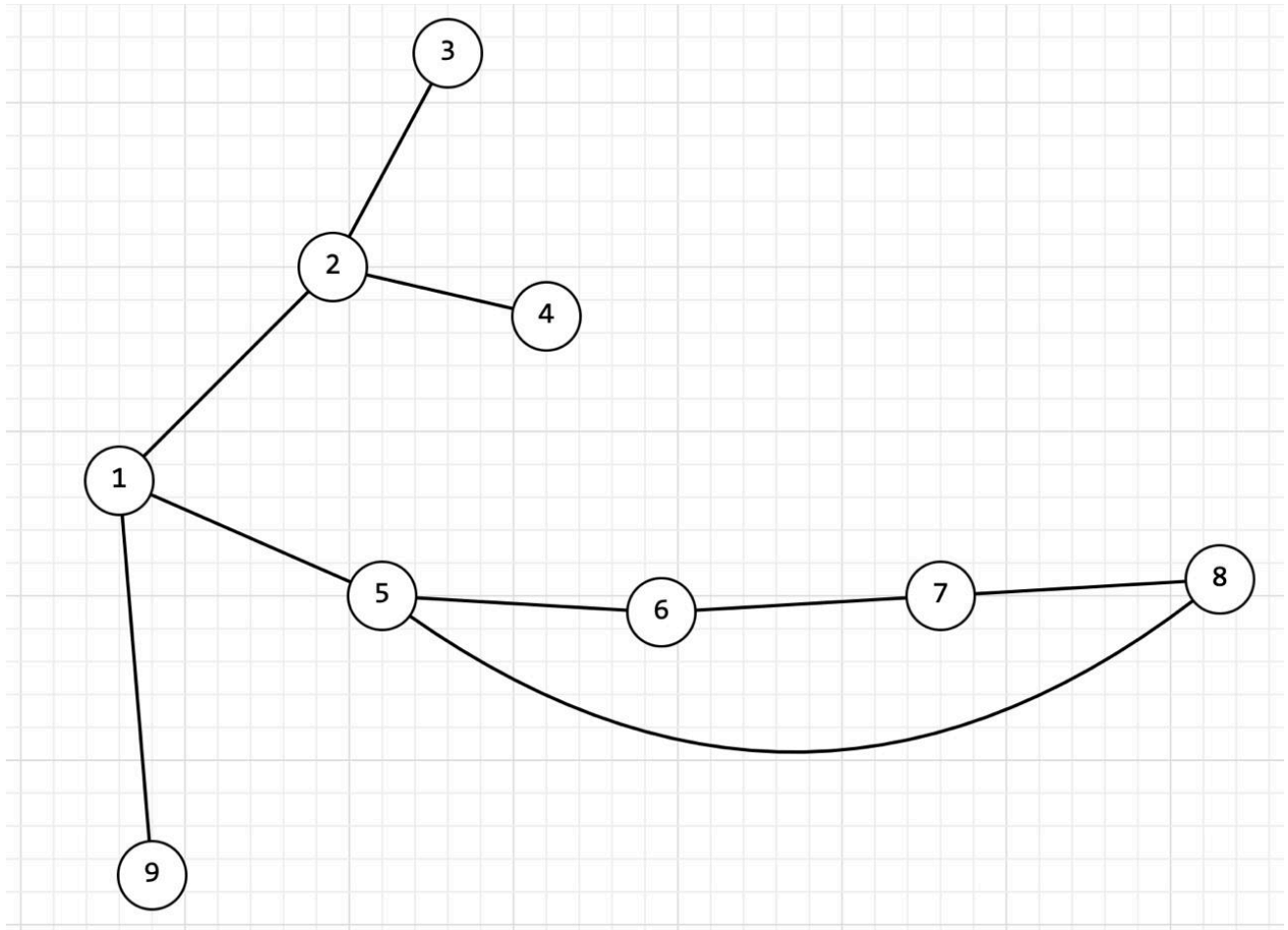
```

```

void dfs(v)
    used[v] = 1; push(s, v);
    while (!empty(s))
        top(s, u);
        pop(s);
        for (w in Adj[u])
            if (!used[w])
                used[w] = 1;
                push(s, w);

```

5. На графе сделать версию обхода в глубину (три цвета) и обхода в ширину (три цвета).



6. Посмотреть время выполнения всех рассмотренных обходов в глубину и в ширину.